

Faculty of Electrical Engineering
Banja Luka
Network and distributed programming

Project Terms of

Reference May 2025.

Create a vehicle service administration system. The system uses a GUI application with options for working with clients (view, add, modify, delete), manage appointments, and order parts from suppliers. In addition to the service application, there are also GUI applications for customers and parts suppliers.

Client application

Clients use a GUI application to interact with the service. After starting, a login form opens with a username and password. In addition, there is an option to register a new account. During registration, personal data is entered (name, surname, vehicle data, address, phone number, e-mail, username and password). All actions are performed by calling the RESTful service endpoint. The account becomes active only after the service provider approves it. After signing up, the client has the following options:

1. **Scheduling an appointment:** Selecting the date, time and type of service (regular service or repair).
2. **Review of service history:** Tabular display of previous appointments with status (completed, cancelled) and a short description.
3. **Chat:** Communication with other clients (1 on 1 and multicast) about experiences with the service through secure sockets, as well as communication with the service technician.

Service application

The service's GUI application enables:

1. **Client management:** Tabular display of all accounts with options to approve registration, block users or delete users. The data is stored in the XML file *clients.xml*.
2. **Appointment management:** View all scheduled appointments, confirm/reject requests, add comments (eg "Battery replacement needed").
3. **Parts management:** CRUD operations for parts stored in the Redis database. Code, name, manufacturer, price, available quantity and description are stored for each part.
4. **Ordering from suppliers:** Sending requests to suppliers for non-existent parts.
The service selects the supplier, enters the quantity and monitors the order status. All parts after the order are registered in the Redis database.
5. **Creating an invoice:** An invoice is a text document that contains information about the parts used and the type of service performed as part of the given service, as well as information about the user and the vehicle.
6. **Sending an invoice:** Sending a document in a ZIP archive to the user's e-mail.

Provider Application Providers

use a GUI application to communicate with the service via sockets. On occasion

startup, the SupplierServer loads a list of several items from one of the online parts sales sites such as: <https://www.pkwteile.de/autoteile/bmw-ersatzteile/3-e90/136829/10130/bremsbelag>. Code, title, price and image (URL) are parsed. To parse the HTML page, use a library like Jsoup or similar. The supplier may:

1. **View your parts:** Table with available parts and prices.
2. **Process orders:** View order queues (oldest first) with options to approve or reject. Approved orders generate an invoice with the amount of VAT (17%) and send it to the accounting service via RMI. Accounts are stored in serialized form on the RMI server.
3. **Update the offer:** Adding new parts via **socket** communication.

Additional specifications

1. All critical events (errors, successful transactions) are logged via Logger.
2. Define the configuration (file paths, ports, URLs) in the config.properties file.
3. Automatic addition of test terms and parts at startup is allowed applications.
4. Each service appointment is fixed for one hour.

Technical requirements

1. Communication: REST (client-service, service-service GUI application), sockets (service-supplier, clients to each other, client-to-servicer), RMI (supplier-bookkeeping), MQ for order queues.
2. Account serialization: JSON or binary format (optional).
3. The system supports one instance of the service application, and multiple client and supplier applications instances.

Note: Elements that are not explicitly defined (eg e-mail structure, GUI layout) can be implemented arbitrarily.

